第 4 章 内存管理

4.1 整体架构

StarryOS 已经具备用户虚拟内存管理的主体框架，支持程序装载、用户堆栈、动态映射、缺页处理和写时复制。Moss 沿用这一设计，重点补全映射语义、权限管理和缺页处理的细节，并优化频繁 `fork` 场景下的 COW 路径。

整体上，系统用分页机制隔离进程，把连续的虚拟地址按页映射到离散的物理内存。程序装载、堆栈和动态映射先确定可用地址范围与访问权限，物理页推迟到首次访问时分配；发生缺页后，内核根据区域用途从匿名内存、文件缓存、共享内存或固定物理地址取得页面。进程复制通过写时复制延后私有页面的实际拷贝。映射解除或进程退出时，内核依据页面所有权和引用关系决定是否写回脏数据、是否释放物理页。

4.2 地址空间结构

内存管理需要同时处理三件事：哪段虚拟地址可以怎样访问、物理页从哪里来、当前已经装入了哪些页面。把它们混在一个结构里，`fork` 时就不得不为每种页面来源写单独的复制逻辑，`mprotect` 也要同时操作区域描述和页表。Moss 把三者分开：`MemoryArea` 只记录区域范围和权限，`Backend` 只负责填页和释放，`PageTable` 只保存已装入的快照。三者职责不交叉，各操作各自只改自己负责的部分。

代码 4-1 展示了三者的包含关系。

```
AddrSpace
├── va_range              可用虚拟地址上下界
├── areas: MemorySet
│   └── BTreeMap<VirtAddr, MemoryArea>
│       ├── va_range      连续的虚拟地址范围
│       ├── flags         读 / 写 / 执行 / 用户态权限
│       └── backend       Linear | Cow | Shared | File
└── pt: PageTable         已装入的虚拟页 → 物理页快照
```

代码 4-1　AddrSpace 三层结构概览

4.2.1 AddrSpace 与 MemoryArea

每个进程通过 `ProcessData` 持有一个 `Arc<Mutex<AddrSpace>>`。同一进程内的线程共享该对象，看到相同的代码段、数据段、堆和共享映射；普通 `fork` 创建新的 `AddrSpace`，带有 `CLONE_VM` 的 clone 则继续共享原地址空间。

`AddrSpace` 的核心字段：

```rust
pub struct AddrSpace {
    va_range: VirtAddrRange,    // 允许用户进程使用的虚拟地址范围
    areas: MemorySet<Backend>, // 按起始地址组织的虚拟内存区域
    pt: PageTable,              // 已生效的虚拟页到物理页映射
}
```

代码 4-2　用户地址空间的核心数据结构

`MemorySet` 是一组互不重叠的 `MemoryArea`，以区域起始地址为键存在 `BTreeMap` 里，便于查找相邻区域和检测重叠。每个 `MemoryArea` 保存虚拟地址范围、访问权限和 `Backend`：

```
pub struct MemoryArea<B: MappingBackend> {
    va_range: AddrRange<B::Addr>, // 一段连续的虚拟地址范围
    flags: B::Flags,              // 读、写、执行等权限
    backend: B,                   // 决定页面从何处取得
}
```

代码 4-3　MemoryArea 的核心字段

区域存在不等于页面已经分配。程序可以拥有一个数百页的匿名区域，而页表中只有实际访问过的几页。这样既保留了连续的虚拟地址范围，也避免为未使用页面占用物理内存。

地址空间操作均围绕这三个字段展开：`map` 增加区域，`unmap` 删除区域和页表项，`protect` 修改权限，`populate_area` 为指定范围准备页面，`handle_page_fault` 处理访问过程中出现的缺页。

4.2.2　Backend 类型

Backend 把不同来源的页面封装为统一接口。`AddrSpace` 不需要了解文件读取、共享页引用和 COW 复制的细节，只在映射、填页、解除映射和复制地址空间时调用 Backend。

```
#[enum_dispatch(BackendOps)]
pub enum Backend {
    Linear(LinearBackend), // 按固定偏移建立虚拟地址到物理地址的映射
    Cow(CowBackend),       // 管理匿名页和私有映射，fork 后执行写时复制
    Shared(SharedBackend), // 让多个地址空间映射同一组物理页
    File(FileBackend),     // 将文件缓存页映射到用户地址空间
}
```

代码 4-4　Backend 的四种类型

Backend 接口中的 `populate` 负责让指定范围满足本次访问权限，`clone_map` 负责地址空间复制。

4.3　页表与地址翻译

4.3.1　多级页表结构

页表把虚拟页号映射到物理页号，并保存读、写、执行和用户态等权限位。Moss 支持 RISC-V64 和 LoongArch64 两种体系结构，各自使用不同的多级页表格式，但通过类型参数共享同一套通用实现。

以 RISC-V64 Sv39 为例，38 位有效虚拟地址按如下位域划分：



图 4-1　Sv39 虚拟地址位域划分

三级页表从 satp 寄存器指向的根页表出发，VPN[2] 索引第一级，VPN[1] 索引第二级，VPN[0] 索引叶子项，最终得到物理页号。LoongArch64 采用四级页表，结构原理相同，仅级数与位域宽度不同。

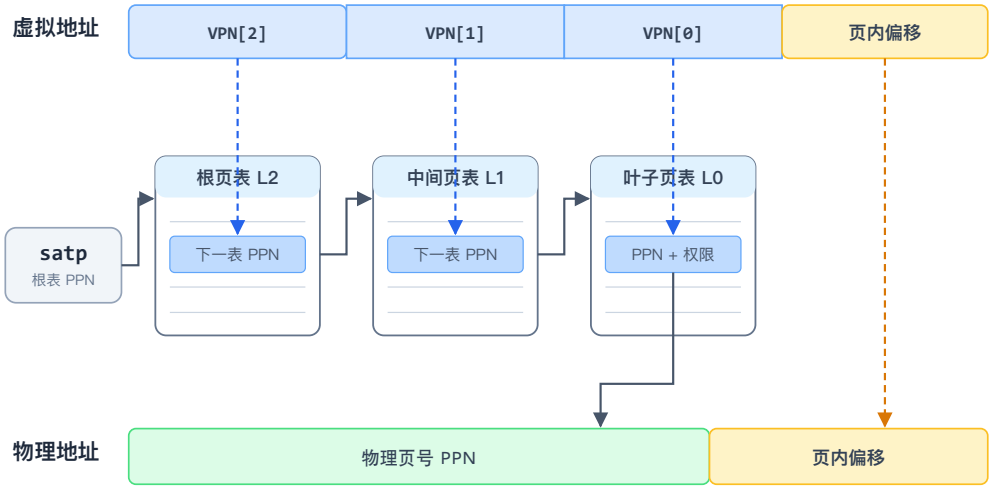


图 4-2　Sv39 三级页表查找过程（4 KiB 叶子页）

axhal 根据编译目标导出对应的 PageTable 类型，上层统一通过 PageTableMut 接口操作，不感知体系结构差异：

```
#[cfg(target_arch = "riscv64")]
pub type PageTable = Sv39PageTable<PagingHandlerImpl>;

#[cfg(target_arch = "loongarch64")]
pub type PageTable = LA64PageTable<PagingHandlerImpl>;

pub struct PageTable64<M, PTE, H> {
    root_paddr: PhysAddr,        // 根页表的物理地址
    _phantom: PhantomData<(M, PTE, H)>,
}
```

代码 4-5　跨体系结构页表的类型组织

页表只保存“当前这一页映射到哪里、允许什么访问”，文件路径、共享关系等信息留在 Backend 中。同一个 MemoryArea 在运行过程中可以经历未映射、只读共享和私有可写等不同页表状态，而 MemoryArea 本身保持不变。

4.3.2　虚拟地址翻译

用户指令给出虚拟地址后，CPU 先查询 TLB。命中时直接得到物理地址，内核不参与。TLB 未命中时，硬件页表遍历器从 satp / CSR 指向的根页表出发，按虚拟地址各级索引逐级查找，取得叶子项后将物理页号写入 TLB，再重新完成访问。

有效页表项提供物理页号和访问权限，原虚拟地址的页内偏移保持不变，最终物理地址由两者拼接而成。若映射存在且权限满足，读写或取指直接在物理内存上完成。

若页表项不存在，或者发生写入只读页面、用户态访问内核页面等权限违规时，CPU 产生缺页异常，将控制权交给内核的缺页处理器。此时缺页并不等于非法访问——它也可能只是页面尚未装入，由 Backend 按需填充后即可重新访问。

4.4 权限管理

进程访问虚拟地址时，权限检查发生在两个层次：硬件在翻译阶段检查页表项标志，软件在缺页时检查 `MemoryArea` 权限。

4.4.1 区域权限与页表权限

`MappingFlags` 定义了四个核心权限位：`READ`、`WRITE`、`EXECUTE` 和 `USER`。`MemoryArea.flags` 记录这段虚拟地址应有的访问权限，是软件维护的意图描述；页表项中的对应位则是硬件实际执行检查的依据。

两层的作用不同：

`MemoryArea.flags` 是区域的规格说明：缺页处理时决定能否向 Backend 请求对应权限的页面，区域权限不允许的访问直接触发 `SIGSEGV`，不进入填页流程。

页表项权限在硬件翻译时强制执行。没有 `USER` 位的页面，用户态代码无法访问，即使虚拟地址落在合法范围内；内核页面因此对用户态始终不可见，不依赖任何软件地址范围检查。`EXECUTE` 位（RISC-V 的 X 位）防止数据页被当作代码执行。

COW 页面在父子进程共享阶段会临时去掉 `WRITE` 位，硬件写访问因此触发缺页，软件再按引用计数决定是复制还是直接恢复权限。权限位的这种临时降级是 COW 机制的触发信号，不代表区域本身变成只读。

4.4.2 mprotect

`mprotect` 在运行时修改已有映射区域的访问权限。内核先检查新权限是否合法——例如，写封印的共享文件映射不允许添加 `WRITE`——再调用 `AddrSpace::protect`。

内部实现复用与 `munmap` 相同的区域切分逻辑，把目标范围从已有区域中划出独立子区域，修改该子区域的 `flags` 并同步更新已装入页的页表项权限。`MemoryArea` 和页表始终保持一致：区域权限改变后，后续缺页以新权限为准，已在页表中的页面也立刻反映新标志。

`PROT_NONE`（全部权限清零）的常见用途是把一段地址划为守卫区。任何访问都会因页表项没有有效权限而在硬件层被拦截，产生 `SIGSEGV`，无需经过 `MemoryArea` 的软件检查。

4.5 地址空间的建立

装载程序时，信号跳板、ELF 各段、用户栈和用户堆共同构成进程的初始虚拟地址空间。图 4-3 给出了 RISC-V64 与 LoongArch64 下代码中明确规定的地址位置；其中 ELF 各段的实际范围由程序头决定，图中只标出加载基址。信号跳板映射到固定用户地址，设置为只读可执行，使用内核已有物理页，删除地址空间时只移除映射、不释放物理页。

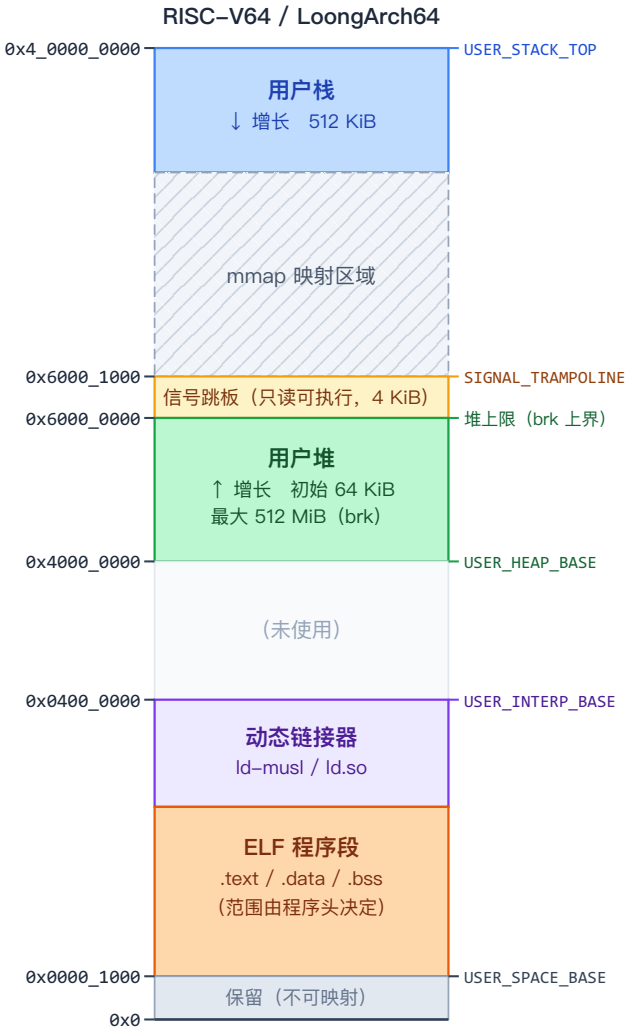


图 4-3　Moss 用户虚拟地址空间布局

4.5.1　ELF 段映射

ELF 文件以“程序段”的形式记录装载说明，类型为 PT_LOAD 的段必须放入用户地址空间。加载器逐个处理可装载段，按 ELF 中记录的权限建立映射：

```
for ph in program_headers.iter().filter(|ph| ph.get_type() == Ok(Type::Load)) {
    let vaddr = ph.virtual_addr as usize + elf_base;
    let seg_start = VirtAddr::from_usize(vaddr);
    let seg_pad = vaddr.align_offset_4k();
    let seg_align_size = align_up_4k(ph.mem_size as usize + seg_pad);

    let backend = Backend::new_cow(
        seg_start,
        PageSize::Size4K,
        FileBackend::Cached(cache.clone()), // ELF 文件的缓存对象
        ph.offset,
        Some(ph.offset + ph.file_size),     // 文件有效数据的结束位置
    );
```

```
    uspace.map(
        seg_start.align_down_4k(),
        seg_align_size,
        mapping_flags(ph.flags),
        false, // 只登记区域，不立即装入物理页
        backend,
    )?;
}
```

代码 4-6　ELF 段的懒加载映射

建立映射时，内核只记录这段地址可以怎样访问，不立即把整个程序读入物理内存。代码第一次执行或数据第一次读取时，Backend 才分配清零页并从 ELF 文件填入内容。文件中没有保存实际数据的 BSS 部分因此自然保持为零。

Moss 在此路径中加入 ELF 头部缓存，减少频繁 `fork/exec` 时重复读取程序头的开销；同时记录文件有效结束位置，避免把文件尾之后的数据当作正常内容。动态链接器、BusyBox 和 shebang 路径兼容保证了比赛镜像中的 musl、glibc 程序和脚本能够进入同一装载流程。

4.5.2　用户栈与堆

用户栈保存函数调用产生的局部状态。程序启动时，加载器还会将命令行参数、环境变量和辅助向量写入栈顶。内核先登记整段可读写栈区，只为启动数据实际占用的页面分配物理内存，其余页面留待首次访问时填充。

用户堆承载运行期间的动态内存分配。Moss 将堆起点固定为 `USER_HEAP_BASE`，装载程序时建立 64 KiB 的初始堆，之后由 `brk` 调整堆顶。堆扩张时，内核为新增范围建立可读写的匿名映射；堆收缩时，解除新堆顶以外的扩展映射。栈和堆均使用 Cow Backend，页面首次访问时清零分配，`fork` 后通过写时复制暂时共享。

4.5.3　动态内存映射

`mmap` 在现有代码段、堆和栈之外建立新的虚拟内存区域。内核收到请求后，先检查映射长度、文件描述符、文件权限和映射类型，对未对齐的起始地址向下取整；固定映射（`MAP_FIXED`）使用调用者给出的地址，普通映射则通过 `find_free_area` 在地址空间中寻找连续空洞。

映射请求	Backend	行为
`MAP_PRIVATE	MAP_ANONYMOUS`	`Cow`	建立私有匿名区域，页面首次访问时清零分配
`MAP_PRIVATE` 文件映射	`Cow`	从文件读取初始内容，写入后只修改私有副本
`MAP_SHARED	MAP_ANONYMOUS`	`Shared`	预留槽位，物理页在首次访问时按需分配
`MAP_SHARED` 文件映射	`File`	多个映射通过文件缓存观察同一份内容
设备物理范围	`Linear`	直接映射设备提供的物理地址

表 4-1　mmap 标志与 Backend 的对应关系

Moss 补充了 MAP_SHARED_VALIDATE、MAP_FIXED_NOREPLACE、MAP_GROWSDOWN 等标志的处理，并加强文件读写权限与 offset 校验。MAP_LOCKED 标志被记录并统计锁定字节数，mlock 接口完成权限和 rlimit 检查；当前不预填页或固定物理页。Moss 还记录每次 mmap 创建的地址范围及共享或私有属性，供 /proc/[pid]/maps 输出相应的 s（共享映射）或 p（私有映射）标记。

mremap 调整已有映射的大小：缩小时解除末尾多余的映射；扩张时优先在原区域后方追加匿名页面填充；后方空间不足且调用者设置了 MREMAP_MAYMOVE 时，内核将映射迁移到新地址——迁移路径通过 MAP_PRIVATE | MAP_ANONYMOUS 重建目标区域并复制原始内容，原有 Backend 类型不随迁移保留。

4.6 按需分页与缺页处理

4.6.1 按需分配策略

程序镜像中大多数代码页在单次运行中不会被触碰，用户栈在实际增长前也不需要物理页。提前分配整段区域既浪费物理内存，也会拖慢 fork 后立即 exec 的路径——复制了大量页面，却在 exec 时立刻丢弃。

AddrSpace::map 的主要工作是创建 MemoryArea。只有 populate 参数为真时才立即调用 populate_area。通常的 ELF 段、用户栈和普通 mmap 都采用懒加载；MAP_POPULATE、栈参数和初始堆会主动填页。

populate_area 按 MemoryArea 边界遍历目标范围，对每一段调用对应 Backend。Backend 最终通过全局页分配器取得物理页，并使用页表接口建立映射。匿名页和 COW 副本被统计为 VirtMem 用途，释放时归还全局页分配器。

4.6.2 不同来源的填页

Cow Backend 同时承担匿名分配和私有文件映射。匿名页面从全局分配器取得后清零；私有文件页面先清零，再读取该虚拟页对应的文件区间。写入私有文件页后，修改只影响进程自己的副本，不回写原始文件。

Shared Backend 为共享匿名映射预留与大小对应的 Option<PhysAddr> 槽位，物理页在首次 populate 时通过 get_or_alloc 按需分配。多个进程映射到同一 SharedPages 对象时，访问同一槽位得到同一物理页。

Linear Backend 根据固定偏移计算物理地址，不进行内存分配，用于将设备提供的物理范围直接接入用户地址空间。

4.6.3 文件页的接入与淘汰同步

File Backend 从文件系统的页缓存取得物理页，将其映射进用户页表。页缓存以“全局索引 + per-file LRU”组织，具体架构和写回策略已在文件系统章节说明；本节只关注内存管理侧的映射建立与一致性保证。

共享文件映射首次缺页时，File Backend 从缓存取得对应页并建立页表映射，同时向缓存注册淘汰监听器：

```
// FileBackend 向页缓存注册回调，在缓存页被驱逐时删除用户页表项
let handle = cache.add_evict_listener({
    let this = Arc::downgrade(self);
```

```
    move |pn, _page| {
        let Some(this) = this.upgrade() else { return };
        this.on_evict(pn, &mut aspace);
    }
});
```

代码 4-7　File Backend 注册缓存淘汰监听器

缓存页被驱逐时，监听器先删除用户页表项，再完成写回和物理页释放。进程再次访问同一虚拟地址时重新缺页，Backend 从缓存取得新页。这一机制保证用户页表始终不持有已回收的物理页引用。

对于连续的顺序缺页，File Backend 还会检测访问模式并触发预读，提前将后续页面装入缓存，具体策略在文件系统章节说明。

4.6.4　缺页处理流程

图 4-4 展示了用户访问虚拟地址的完整处理路径。映射存在且权限满足时，CPU 直接完成访问；否则进入缺页处理，先查 `MemoryArea` 确认地址和访问类型是否合法，再调用该区域绑定的 Backend 准备页面。

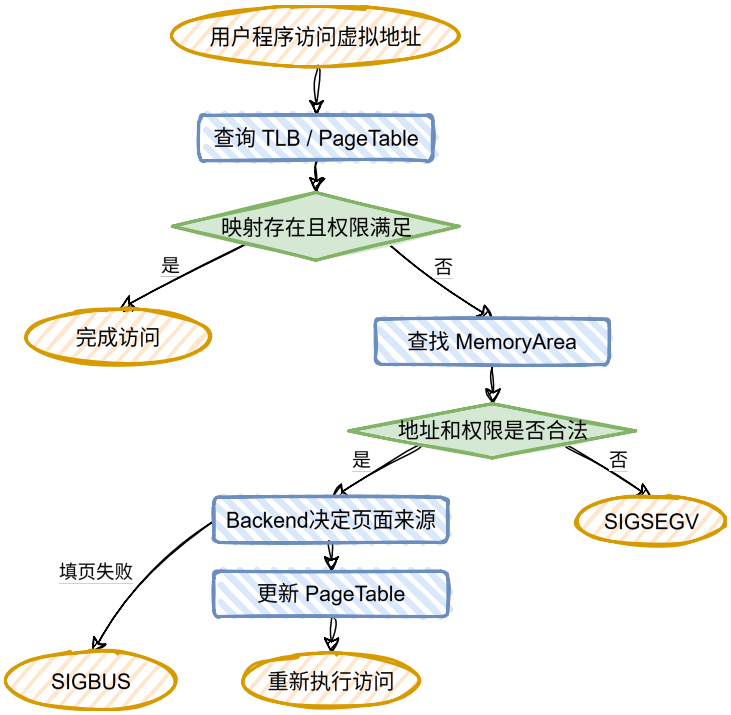


图 4-4　用户访问虚拟地址与缺页处理流程

缺页处理集中在 `AddrSpace::handle_page_fault`：

```
if let Some(area) = self.areas.find(vaddr) {
    let flags = area.flags();
    // 区域权限必须允许本次访问类型
    if flags.contains(access_flags) {
        let page_size = area.backend().page_size();
        return area.backend()
            .populate(
```

```
                VirtAddrRange::from_start_size(
                    vaddr.align_down(page_size),
                    page_size as usize,
                ),
                flags,
                access_flags,
                &mut self.pt.modify(),
            )
            .is_ok();
    }
}
false // 地址不存在或权限不匹配
```

代码 4-8　缺页处理的主体判断

填页成功后，用户上下文返回原指令，CPU 重新执行。若另一个线程已提前建立了相同映射，Backend 会检查现有页表权限并将该页视为已填充。

4.6.5 缺页失败与信号

用户任务循环根据缺页原因选择信号：

情况	处理结果	含义
页面尚未分配，区域和权限均有效	重新填页	正常的按需分页
地址落在 grow-down 预留范围	扩展映射	为向下增长区域增加一页
地址没有对应区域或权限不符	SIGSEGV	进程访问了无效虚拟地址
合法文件映射无法提供页面	SIGBUS	地址有效，但后端存储对象不能完成访问

表 4-2　用户缺页的处理结果

Moss 补全了 MAP_GROWSDOWN 的缺页扩展记录，并为部分文件映射和 mremap 场景维护 SIGBUS 区域。LoongArch64 用户态非对齐异常还会先尝试软件模拟，无法完成时再转为 SIGBUS。

4.7 地址空间复制与写时复制

4.7.1 fork 时的地址空间处理

普通 fork 调用 AddrSpace::try_clone 创建新地址空间。内核先复制父进程的虚拟区域描述，使子进程拥有相同的代码、数据、堆、栈和映射布局；随后处理已装入页表的页面。对于私有可写页面，父子进程暂时共享同一物理页，页表权限降为只读。fork 不需要立即复制整个进程内存，真正的数据复制推迟到后续写入时完成。

文件映射和共享映射在这一阶段保留原有的共享关系：文件映射继续使用同一文件缓存来源，并为子地址空间注册独立的淘汰监听器；共享内存继续指向同一组物理页。子进程因此得到与父进程一致的虚拟布局，但是否共享物理页由映射类型和后续访问决定。

带有 CLONE_VM 的 clone 不进入上述过程，而是让新任务直接持有原 AddrSpace，这也是同一进程内线程共享内存的实现基础。

4.7.2 COW 写缺页

父子进程共享 COW 页面后，任意一方写入都因页表缺少 `WRITE` 权限而触发缺页。Cow Backend 查询物理页引用计数：

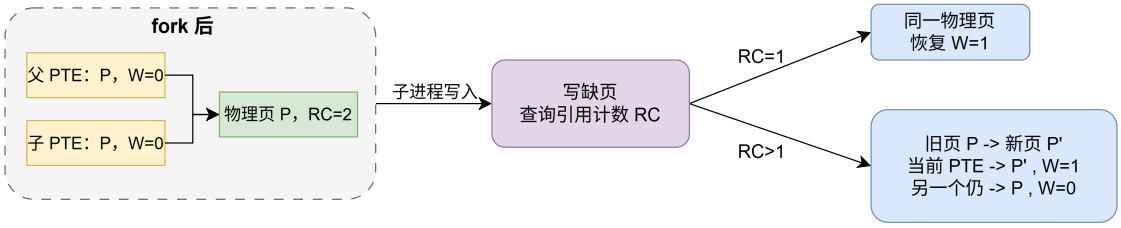


图 4-5　COW 的共享与写时复制

```
match frame_ref_count {
    1 => {
        // 已无其他地址空间共享，直接恢复写权限
        pt.protect(vaddr, flags)?;
    }
    _ => {
        // 仍被共享，复制数据并改映射到新页面
        let new_frame = alloc_new_frame(false)?;
        copy_page(paddr, new_frame);
        pt.remap(vaddr, new_frame, flags)?;
        drop_old_reference(paddr);
    }
}
```

代码 4-9　COW 写缺页的两种处理

引用数为一说明其他共享者已退出或完成复制，当前进程可直接恢复写权限。引用数大于一时，内核分配新页并复制原数据，只修改当前进程的页表。父子进程从此拥有内容相同但相互独立的页面。

4.7.3 COW 路径优化

StarryOS 原有 COW 引用计数范围较小，大量 `fork` 可能造成计数溢出。Moss 将计数扩展为 `usize`，并在增加引用前检查溢出。

Moss 还为 Cow Backend 记录已装入的页面数量。一个懒加载区域如果从未发生缺页，父页表中就没有需要共享的物理页，`fork` 可以直接复制区域描述，跳过逐页查询。对于已经装入的页面，才执行只读保护、引用计数增加和子页表映射。这项修改减少了频繁 `fork/exec` 场景中的无效页表遍历。

4.8 地址空间的解除与释放

区域删除的主路径由 `munmap` 触发。`MemorySet` 内部用有序结构保存互不重叠的 `MemoryArea`，删除一段范围时不需要逐页扫描整个地址空间。它先删除完全落在目标范围内的区域，再处理左右边界上的相交区域：覆盖区域尾部时收缩右端，覆盖头部时收缩左端，位于中间时把原区域切成左右两个。每次区域收缩或删除时，底层页表映射都会同步解除。

```rust
pub fn unmap(
    &mut self,
    start: B::Addr,
    size: usize,
    page_table: &mut B::PageTable,
) -> MappingResult {
    let range = AddrRange::try_from_start_size(start, size)
        .ok_or(MappingError::InvalidParam)?;

    // 删除完全位于目标范围内的 MemoryArea
    self.areas.retain(|_, area| {
        if area.va_range().contained_in(range) {
            area.unmap_area(page_table).unwrap();
            false
        } else {
            true
        }
    });

    // 左右边界的相交区域：收缩或切分，各自同步清除页表项
    ...
}
```

代码 4-10　MemorySet 的区域删除逻辑

`mprotect` 与 `mremap` 复用了这套区域切分能力：`mprotect` 不删除区域，只在切分后修改目标子区域的权限和页表标志；`mremap` 缩小时调用 `unmap` 删除末尾范围，扩张时优先在原区域后方追加匿名填充，空间不足且允许移动时迁移到新地址。`brk` 收缩也会进入区域删除路径，但只解除初始堆区之外的扩展页面。`execve` 和地址空间析构调用 `AddrSpace::clear`，一次性删除全部区域。

`AddrSpace` 实现了 `Drop`，进程最后一个地址空间引用消失时自动调用 `clear`。因此，正常退出、创建失败后的对象回收和 exec 替换最终都汇入同一套区域删除与 Backend 释放流程。

4.9　本章小结

Moss 的内存管理以 `AddrSpace`、`MemoryArea`、`PageTable` 和 Backend 四个结构为核心。地址空间先登记可用区域，页面由 Backend 按需装入；多级页表负责虚拟地址翻译，`USER` 和 `EXECUTE` 等权限位在硬件层隔离用户态访问；缺页处理连接区域描述和物理页面；`mprotect` 在运行时修改区域权限并同步页表；`fork` 根据 Backend 类型选择共享、COW 或线性重建，私有可写页面通过引用计数驱动的 COW 延后复制。

Moss 在 StarryOS 基础上补全了若干细节：`mmap` 对未对齐起始地址向下取整；`MAP_LOCKED` 接受请求并统计锁定字节数；`mremap` 迁移路径通过匿名私有映射重建，原 Backend 不随迁移保留；`msync` 目前只作为接口桩存在，脏页写回依赖 `fsync` 和全局 `sync` 触发；COW 引用计数扩展为 `usize` 并加入溢出检查。文件页在用户页表中的接入与淘汰同步由 File Backend 负责，缓存架构和写回策略已在文件系统章节说明。